

CUDA Conv2d

MGP 2025 Spring

Due: 06 APR (SUN) 23:59

Hello, MGP class!

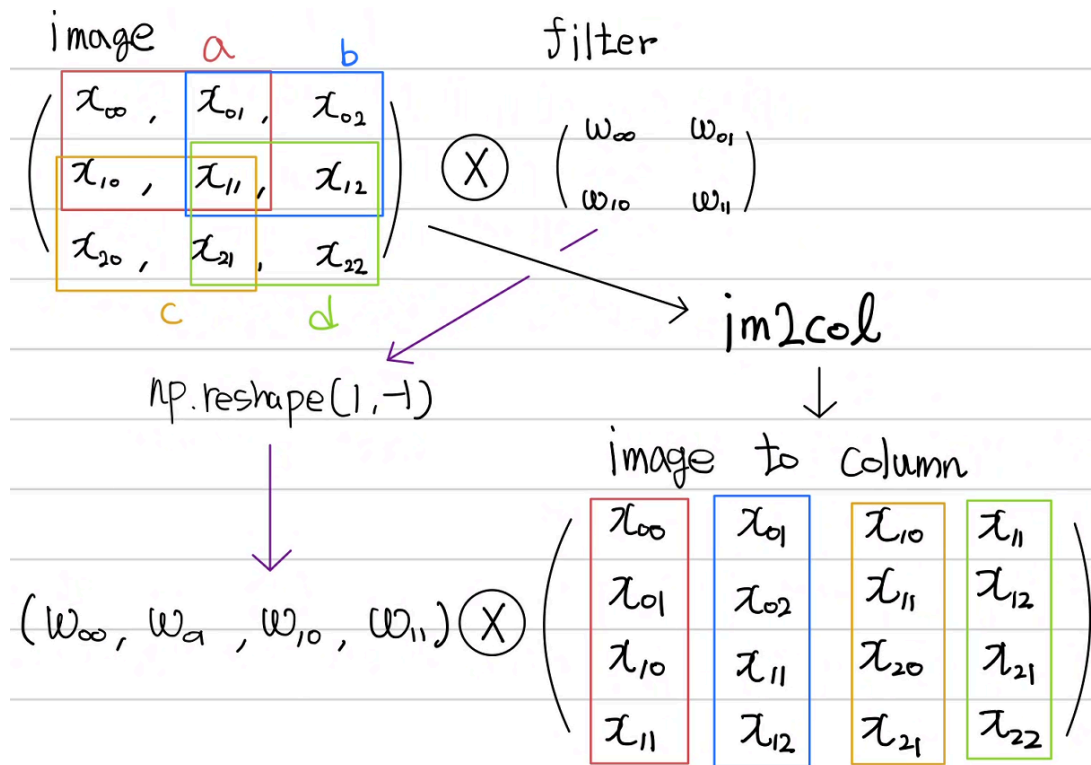
This assignment challenges you to harness the power of GPU parallelism by implementing a critical operation in two different ways:

1. Parallel Conv2d:

In this approach, you'll implement the convolution operation directly on the GPU using CUDA.

2. Parallel im2col combined with Matmul:

This approach transforms the convolution operation into a matrix multiplication problem, which can be highly optimized on modern hardware. The im2col (image-to-column) technique restructures the input image data by extracting overlapping patches and flattening them into columns of a 2D matrix.



The goal is to optimize these operations for both correctness and high performance. You'll gain hands-on experience in designing efficient CUDA kernels and optimizing memory access patterns, which are essential skills in high-performance computing.

This project not only tests your technical abilities but also encourages innovative solutions to meet strict performance benchmarks.

[Overview]

You are required to implement three functions in `conv.h`. The input size will remain fixed during grading, although the values within the input may vary. **The batch size is always 1.** You should carefully reconsider how the filter size may affect your implementation.

[Code - 10 pts]

The total is 10 pts.

- 2 pts, if your both programs work. No partial points.
- 2 pts, if your parallel **Conv2d with filter size 3x3** runs within **0.3 sec.**
- 2 pts, if your parallel **im2col+GEMM with filter size 3x3** runs within **1.5 sec.**
- 2 pts, if your parallel **Conv2d with filter size 7x7** runs within **0.3 sec.**
- 2 pts, if your parallel **im2col+GEMM with filter size 7x7** runs within **6.0 sec.**

Bonus points are awarded if your program outperforms and is not a late submission.

- 2 pt, if your parallel **Conv2d with filter size 3x3** runs faster than **0.15 sec.**
- 2 pt, if your parallel **im2col+GEMM with filter size 3x3** runs faster than **0.3 sec.**

[Report - 10 pts]

You have to submit an "excellent" report about your homework, including why the im2col+GEMM approach does not show the expected performance in this homework. If you reveal the exact reason, you will be awarded a coffee ☕.

[Note]

Your code's speed is measured based on the maximum of 5 executions on the grading server. I won't regrade due to "It worked on my machine" excuses, so I encourage you to perform the assignment more generously than the criteria. As many students tend to rush near the deadline, your code may appear slower in the local environment. Therefore, I recommend completing the assignment **as early as possible** when the server is less

congested.

[Rules]

1. Put your HW4 directory under `<your_home>/mgp/hw4`
 - a. All your make commands, especially 'make run' should work.
2. **Do not change any** of the output(i.e. 'std::cout') statements. We will grade based on the program outputs. Any change on the outputs would probably cause an error, leading to 0 points for your HW assignment.
3. Don't touch anything in the directory after the deadline. If any of the timestamps are later than the deadline, we will treat it as 'late submission'.
4. You can't use *CUDA* libraries for this assignment.
5. Please retain adequate permission on your files to avoid plagiarism.
6. No abusing. Please use your common sense.
7. Zero GPT policy. No mercy.